

Mock Final Exam

CS 101 L4 Algorithmic Problem Solving

Fall 2023

1. Min-max

Write a *recursive* function, `minmax`, that takes as argument a `list` called `lst` that contains `int` values, and returns a `tuple` containing the smallest and largest element respectively in the `list`.

Constraints

- `isinstance(lst, list) and len(lst) > 0`
- `all(isinstance(n, int) for n in lst)`

Sample

Input	Output
[1]	(1, 1)
[0, -3, 6, 4, 9, 1]	(-3, 9)

Solution:

```
1 def minmax(lst):
2     # Base case - a single element is the min and the max.
3     if len(lst) == 1:
4         return (lst[0], lst[0])
5     # Recursion: compare current element with min and max of remaining
6         list.
7     small, large = minmax(lst[1:])
8     if lst[0] < small:
9         small = lst[0]
10    if lst[0] > large:
11        large = lst[0]
12    return small, large
```

2. Dot

Given 2 *vectors*, each of size n ,

$$A = \begin{bmatrix} a_1 \\ a_2 \\ \dots \\ a_n \end{bmatrix}, B = \begin{bmatrix} b_1 \\ b_2 \\ \dots \\ b_n \end{bmatrix}$$

their *dot product* is defined as $a_1b_1 + a_2b_2 + a_3b_3 + \dots + a_nb_n$.

Write a *recursive* function, `dot`, that takes the vectors A and B, each represented as a `list`, and returns their dot product.

Constraints

- `isinstance(A, list)` and `isinstance(B, list)`
- `len(A) == len(B)`
- `len(A) > 0`
- `all(isinstance(n, int) for n in A)`
- `all(isinstance(n, int) for n in B)`

Sample

Input	Output
[1], [3]	3
[0, -3, 6], [4, 9, 1]	-21

Solution:

```
1 def dot(A, B):
2     # Base case - Vectors are empty, dot product is 0.
3     if A == []:
4         return 0
5     # Recursion: product of current elements + dot product of remaining
6     #               vectors.
7     return A[0] * B[0] + dot(A[1:], B[1:])
```

3. Pizza Time

You want to order 3 custom toppings for your pizza and have a budget of B rupees. `toppings` is a `dict` containing names of available topping as keys and the price of each as values. You can select a combination of 3 distinct toppings whose combined price does not exceed B.

Write a function, `num_toppings`, that takes `toppings` and B as arguments and returns the *number of possible combinations* that you can order.

Constraints

- `isinstance(B, int) and B > 0`
- `isinstance(toppings, dict) and len(toppings) >= 3`
- `all(isinstance(k, string) for k in toppings.keys())`
- `all(isinstance(v, int) for v in toppings.values())`

Sample

Input	Output
<code>{'olives': 250, 'chicken': 350, 'onions': 150, 'tomatoes': 200, 'pineapple': 300}</code> 700	4

There are 4 possible combinations whose combined price does not exceed 700:

Combination	Price
(olives, onions, tomatoes)	600
(olives, onions, pineapple)	700
(chicken, onions, tomatoes)	700
(onions, tomatoes, pineapple)	650

Solution:

```
1 def num_toppings(prices, budget):
2     # Keys are not required - only interested in the prices.
3     prices = list(prices.values())
4     m = len(prices)
5     pizzas = 0
6     # Iterate over all unique triples, count those that satisfy the
7         condition.
8     for i in range(m):
9         for j in range(i+1, m):
10            for k in range(j+1, m):
11                if prices[i] + prices[j] + prices[k] <= budget:
12                    pizzas += 1
13
14     return pizzas
```

4. Recursive Pizza Time

You now want to order n distinct toppings for your pizza in a budget of B rupees. `prices` is a `list` containing the prices of the available toppings.

Write a *recursive* function, `num_n_toppings`, that takes `prices`, B , and n as arguments and returns the *number of possible combinations* that can be ordered.

Constraints

- `isinstance(B, int) and B > 0`
- `isinstance(n, int) and n > 0`
- `isinstance(prices, list) and len(prices) > 0`
- `all(isinstance(p, int) for p in prices)`

Sample

Input	Output
[250, 350, 150, 200, 300] 700 3	4
[250, 350, 150, 200, 300] 500 2	6

The first sample is as in the previous problem.

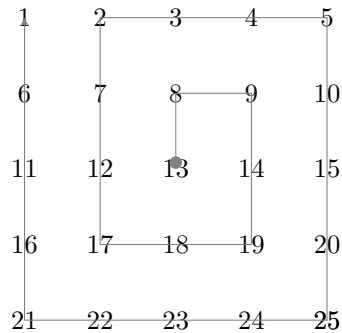
In the second sample, we want 2 toppings in a budget of 500 rupees. There are 6 possible pairs of toppings: (olives, onions), (olives, tomatoes), (chicken, onions), (onions, tomatoes), (onions, pineapple), and (tomatoes, pineapple).

Solution:

```
1 def num_n_toppings(prices, budget, n):
2     # More toppings required than available - not possible.
3     if n > len(prices):
4         return 0
5     # Base case - no toppings required - only one way to satisfy.
6     elif n == 0:
7         return 1
8     # Recursion - take a suitable topping, and find combinations in the
9         # remaining toppings.
10    num = 0
11    for i in range(len(prices)):
12        p = prices[i]
13        if p <= budget:
14            num += num_n_toppings(prices[i+1:], budget-p, n-1)
15    return num
```

5. Spiral

We are given a square matrix, `M`, of size `n` as a nested `list`. Each inner `list` represents a row of `M`. We want to traverse all the elements of `M` in spiral order starting from its center element, as pictured below.



This is a matrix with `n==5`. The traversal starts at the center element of the matrix and spirals outward. The traversal is the list of elements visited in this order. Namely, `[13, 8, 9, 14, 19, 18, 17, 12, 7, 2, 3, 4, 5, 10, 15, 20, 25, 24, 23, 22, 21, 16, 11, 6, 1]`. Write a function, `spiral`, that takes a square matrix, `M`, as described above. It returns a list containing all the elements of `M` in spiral order.

Constraints

- `isinstance(M, list)`
- `len(M) > 0 and len(M)%2 == 1`
- `all(isinstance(row, list) and len(row)==len(M) for row in M)`

Sample

Input	Output
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]	[5, 2, 3, 6, 9, 8, 7, 4, 1]
[[9]]	[9]

Solution:

```
1 def spiral(matrix):
2     '''
3     After visiting the middle elements, the traversal repeats the
4     following
5
6     order: UP, RIGHT, DOWN, LEFT.
7
8     The length of each move in these directions is:
9     1, 1, 2, 2, 3, 3, 4, 4, ... , n, n
10    followed by a last step of length n.
11    '''
12    n = len(matrix)
13    # Length in each direction after including the middle element.
14    lengths = []
15    for i in range(1, n):
16        lengths += [i, i]
17    moves.append(n-1) # last move of length n
18    # Store middle element.
19    row = col = n // 2 # indexes in spiral order, start with mid
20    spiral = [matrix[row][col]] # visited elements so far
21    direction = 0 # 0:UP, 1:RIGHT, 2:DOWN, 3:LEFT
22    # Make the generated moves and store the visited elements.
23    for length in lengths:
24        for _ in range(length):
25            # Adjust index according to direction
26            if direction == 0: # UP
27                row -= 1
28            elif direction == 1: # RIGHT
29                col += 1
30            elif direction == 2: # DOWN
31                row += 1
32            else: # LEFT
33                col -= 1
34            spiral.append(matrix[row][col]) # store element
35            direction = (direction + 1) % 4 # next direction
36    return spiral
```